

```

<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AIカフェ店員</title>
  <style>
    body { font-family: 'Helvetica Neue', Arial, sans-serif; display: flex;
flex-direction: column; align-items: center; justify-content: center; height: 100vh;
background: #f5efe6; color: #4f46e5; margin: 0; }
    .container { background: #ffffff; padding: 40px; border-radius: 24px;
box-shadow: 0 10px 30px rgba(0,0,0,0.05); text-align: center; width: 90%; max-width:
400px; border: 2px solid #e1d7c6; }
    h2 { color: #8b5a2b; margin-bottom: 5px; }
    p { color: #6e655f; font-size: 14px; margin-bottom: 25px; }
    button { background: #8b5a2b; color: white; border: none; padding: 16px 32px;
font-size: 18px; border-radius: 30px; cursor: pointer; transition: 0.3s; font-weight:
bold; width: 100%; box-shadow: 0 4px 12px rgba(139,90,43,0.3); }
    button:hover { background: #6f4722; }
    button:disabled { background: #cccccc; cursor: not-allowed; box-shadow: none; }
    #status { margin-top: 25px; font-size: 15px; color: #8b5a2b; font-weight: 500;
}
    #log { margin-top: 20px; text-align: left; background: #faf8f5; padding: 12px;
border-radius: 12px; max-height: 150px; overflow-y: auto; font-size: 13px; color: #555;
border: 1px solid #eee; }
  </style>
</head>
<body>

<div class="container">
  <h2>MinaRAI カフェAI店員</h2>
  <p>ボタンを押して注文を話しかけてください</p>
  <button id="talk-btn">話しかける</button>
  <div id="status">ボタンを押すと注文が始まります</div>
  <div id="log"><strong>注文履歴:</strong><br></div>
</div>

<script>
  // ——— 【修正】GitHub Actionsが本物のキーに置き換える正しい記述 ———
  const GEMINI_API_KEY = "__GEMINI_API_KEY_PLACEHOLDER__";
  const GEMINI_URL =
`https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateConte
nt?key=${GEMINI_API_KEY}`;

  const SYSTEM_INSTRUCTION = `
あなたはオシャレなカフェの若い女性店員「ミナライ」です。
【性格・特徴】
- 明るく、丁寧で、親しみやすい接客をしてください。
- 返答は1〜2文程度で、極めて「簡潔」にテンポよく話してください（長文は厳禁）。
- ユーザーから「コーヒー」「お茶」「カフェオレ」のいずれかの注文を聞き出してくださ
い。
- 注文を受け付けたら、「かしこまりました！（注文されたもの）ですね、少々お待ちくだ
さい！」のように明るくハキハキと答えて会話を締めくくってください。
`;

  // ♡ 履歴の管理方法をGeminiの標準リクエスト規約に完全準拠させました
  let conversationHistory = [];

  const talkBtn = document.getElementById('talk-btn');
  const statusDiv = document.getElementById('status');
  const logDiv = document.getElementById('log');
  let hasOrdered = false;

  // 音声認識の初期化

```

```

const SpeechRecognition = window.SpeechRecognition ||
window.webkitSpeechRecognition;
const recognition = new SpeechRecognition();
recognition.lang = 'ja-JP';
recognition.interimResults = false;
recognition.maxAlternatives = 1;

talkBtn.addEventListener('click', () => {
  window.speechSynthesis.cancel();
  try {
    recognition.start();
    statusDiv.innerText = "ご注文をどうぞ！聞き取り中...";
    talkBtn.disabled = true;
  } catch (e) {
    console.error("開始エラー:", e);
  }
});

recognition.onresult = async (event) => {
  let userText = "";
  try {
    userText = event.results[0][0].transcript; // 正確な多重配列アクセス
  } catch (e) {
    console.error("テキスト抽出に失敗:", e);
  }

  if (!userText || userText.trim() === "") {
    statusDiv.innerText = "うまく聞き取れませんでした。";
    talkBtn.disabled = false;
    return;
  }

  addLog(`あなた: ${userText}`);
  statusDiv.innerText = "店員が考えています...";

  // ユーザーの発言を履歴に追加
  conversationHistory.push({ role: "user", parts: [{ text: userText }] });

  const aiResponse = await fetchAIResponse();
  addLog(`店員: ${aiResponse}`);

  checkAndDownloadOrder(aiResponse);
  speak(aiResponse);
};

recognition.onend = () => {
  if (statusDiv.innerText === "ご注文をどうぞ！聞き取り中...") {
    statusDiv.innerText = "次の注文をどうぞ（ボタンを押してください）";
    talkBtn.disabled = false;
  }
};

// 💡 【大修正】 URLとBodyの構造をGemini APIの正式仕様に完全修正
async function fetchAIResponse() {
  try {
    const response = await fetch(GEMINI_URL, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        contents: conversationHistory,
        // システム命令をAPI仕様どおりに独立して注入
        systemInstruction: {
          parts: [{ text: SYSTEM_INSTRUCTION }]
        }
      })
    });
  }
}

```

```

    });
    const data = await response.json();

    // 安全にデータを抽出
    if (data.candidates && data.candidates[0] && data.candidates[0].content &&
data.candidates[0].content.parts && data.candidates[0].content.parts[0]) {
        const aiText = data.candidates[0].content.parts[0].text;
        conversationHistory.push({ role: "model", parts: [{ text: aiText }] });
        return aiText;
    } else {
        console.error("想定外のレスポンス構造:", data);
        return "すみません、もう一度おっしゃっていただけますか?";
    }
} catch (error) {
    console.error("API通信エラー:", error);
    return "すみません、通信エラーが起きてしまいました。もう一度お試しください。";
}

function checkAndDownloadOrder(text) {
    if (hasOrdered) return;

    let orderItem = "";
    if (text.includes("コーヒー")) { orderItem = "コーヒー"; }
    else if (text.includes("お茶")) { orderItem = "お茶"; }
    else if (text.includes("カフェオレ")) { orderItem = "カフェオレ"; }

    if (orderItem) {
        hasOrdered = true;
        addLog(`☑ 検知: ${orderItem}`);

        const blob = new Blob([orderItem], { type: "text/plain;charset=utf-8" });
        const a = document.createElement("a");
        a.href = URL.createObjectURL(blob);
        a.download = "order.txt";
        document.body.appendChild(a);
        a.click();
        document.body.removeChild(a);

        addLog(`☑ order.txt を自動保存しました`);
    }
}

function speak(text) {
    statusDiv.innerText = "店員が話しています...";
    const utterance = new SpeechSynthesisUtterance(text);
    utterance.lang = 'ja-JP';

    // ♪ Windows / Chrome環境で利用可能な日本語の声リストを取得
    const voices = window.speechSynthesis.getVoices();

    // 【選択肢A】 Google Chrome標準の、最も滑らかで自然な女性の声（おすすめ）
    let selectedVoice = voices.find(v => v.lang.includes('ja') &&
v.name.includes(' Google'));

    // 【選択肢B】 もしGoogleの声がなければ、Windows標準の女性の声「Haruka」を試す
    if (!selectedVoice) {
        selectedVoice = voices.find(v => v.lang.includes('ja') &&
v.name.includes(' Haruka'));
    }

    // 【選択肢C】 それもなければ、最初に見つかった日本語の声割り当てる

```

```

    if (!selectedVoice) {
        selectedVoice = voices.find(v => v.lang.includes('ja'));
    }

    // 選んだ声を適用
    if (selectedVoice) {
        utterance.voice = selectedVoice;
        console.log("🔊 使用中の音声:", selectedVoice.name); // 開発画面に声の名前
を表示
    }

    // 💡 声の高さとスピードの微調整 (ここでキャラクター感をさらに演出できます)
    utterance.pitch = 1.1; // 声の高さ (0.5 ~ 2.0 / 少し高めにして若い女性らし
<)
    utterance.rate = 1.05; // 話す速度 (0.1 ~ 10.0 / 少し早めにしてハキハキと)

    utterance.onend = () => {
        if (hasOrdered) {
            statusDiv.innerText = "ありがとうございました！またお待ちしております。";
            talkBtn.disabled = true;
        } else {
            statusDiv.innerText = "次の注文をどうぞ (ボタンを押してください)";
            talkBtn.disabled = false;
        }
    };

    window.speechSynthesis.speak(utterance);
}

function addLog(msg) {
    logDiv.innerHTML += msg + "<br>";
    logDiv.scrollTop = logDiv.scrollHeight;
}

window.speechSynthesis.onvoiceschanged = () => {};
</script>

</body>
</html>

```